

SIMATS ENGINEERING
Department of Computer Science &
Engineering ASSESSMENT TOOL 1- ANALYTICAL
PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Giftson Kevin Matthew.Y	Reg.No.:	192511167
Department:	B.Tech (CSE)	Date:	25\07\2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why

non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I Giftson Kevin Matthew. Y (192511167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Giftson Kevin Matthew.Y

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 1:

Step 1: Convert the Decimal Numbers into Binary

Representation of +45

Decimal number = **45**

Binary of 45:

$$45 = 32 + 8 + 4 + 1$$

Therefore,

$$45 = 00101101$$

(8-bit representation)

Decimal	Binary
+45	00101101

Representation of -23

First write +**23** in binary.

$$23 = 16 + 4 + 2 + 1$$

Therefore,

$$23 = 00010111$$

Now find the **2's complement**.

Step 1: Find 1's Complement

00010111

↓

11101000

Step 2: Add 1

11101000

+00000001

11101001

Therefore,

-23 = 11101001

Decimal	Binary
-23	11101001

Step 2: Addition (+45 + (-23))

Perform binary addition.

00101101 (+45)

+ 11101001 (-23)

1| 00010110

The carry generated outside the 8-bit register is ignored.

Final 8-bit result:

00010110

Convert to decimal.

00010110

=16+4+2

=22

Therefore,

$$45 + (-23) = 22$$

Overflow Check

Overflow occurs only when

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Here,

One number is positive.

The other number is negative.

Therefore,

Overflow cannot occur.

Hence,

No Overflow

Processor Interpretation

The processor observes that the Most Significant Bit (MSB) is **0**.

MSB = 0 indicates a positive number.

Binary result

00010110

represents

22

Therefore, the processor interprets the answer as

+22

Step 3: Subtraction (+45 – (–23))

Subtracting a negative number is equivalent to adding its positive value.

Therefore,

$$45 - (-23)$$

$$=45 + 23$$

Binary Representation

$$+45$$

$$00101101$$

$$+23$$

$$00010111$$

Now perform addition.

$$\begin{array}{r} 00101101 \\ + 00010111 \\ \hline 01000100 \end{array}$$

Convert Result into Decimal

$$01000100$$

$$=64+4$$

$$=68$$

Therefore,

$$45 - (-23) = 68$$

Overflow Check

Both operands are positive.

The result is also positive.

The value 68 lies within the range of an 8-bit signed integer.

Range:

-128 to +127

Since

$68 < 127$

there is

No Overflow

Processor Interpretation

MSB = 0

Hence the processor identifies the result as a positive number.

Binary

01000100

is interpreted as Decimal 68

How the Processor Detects Overflow

The processor checks the carry into the sign bit and the carry out of the sign bit.

Overflow occurs when these two carries are different.

Another simple rule is:

- Positive + Positive $\rightarrow$ Negative $\rightarrow$ Overflow
- Negative + Negative $\rightarrow$ Positive $\rightarrow$ Overflow
- Positive + Negative $\rightarrow$ Never Overflow

In this problem,

Addition:

Positive + Negative

No Overflow

Subtraction:

Positive + Positive

Positive Result

No Overflow

Hence,

Overflow does not occur in either operation.

Final Results

Operation	Binary Result	Decimal Result	Overflow
$+45 + (-23)$	00010110	22	No
$+45 - (-23)$	01000100	68	No

Interpretation of Results

The processor correctly performs both addition and subtraction using 2's complement representation. The results obtained are 22 and 68, with no overflow since both are within the valid 8-bit signed range (-128 to $+127$). The processor uses the MSB to determine the sign and checks the carry bits to detect overflow. Therefore, both operations are performed accurately.

Question 2:

Step 1: Given Binary Inputs

Let the two 4-bit binary numbers be:

A = 1011

B = 0110

Initial Carry,

C₀ = 0

Bit Position	A	B
A ₃ B ₃	1	0
A ₂ B ₂	0	1
A ₁ B ₁	1	1
A ₀ B ₀	1	0

Step 2: Generate (G) and Propagate (P) Signals

The CLA uses two signals for every bit.

Generate Signal (G)

A carry is generated when both input bits are 1.

Formula:

$$G = A \times B$$

Propagate Signal (P)

A carry is propagated when either input bit is 1.

Formula:

$$P = A \oplus B$$

Now calculate each bit.

Bit	A	B	Generate (G=A·B)	Propagate (P=A⊕B)
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Therefore,

$$G_0 = 0$$

$$G_1 = 1$$

$$G_2 = 0$$

$$G_3 = 0$$

and

$$P_0 = 1$$

$$P_1 = 0$$

$$P_2 = 1$$

$$P_3 = 1$$

Step 3: Carry Computation

Unlike the Ripple Carry Adder, the CLA computes all carries in advance.

The carry equations are:

$$C_1 = G_0 + (P_0 \times C_0)$$

$$C_2 = G_1 + (P_1 \times G_0) + (P_1 \times P_0 \times C_0)$$

$$C_3 = G_2 + (P_2 \times G_1) + (P_2 \times P_1 \times G_0) + (P_2 \times P_1 \times P_0 \times C_0)$$

$$C_4 = G_3 + (P_3 \times G_2) + (P_3 \times P_2 \times G_1) + (P_3 \times P_2 \times P_1 \times G_0) + (P_3 \times P_2 \times P_1 \times P_0 \times C_0)$$

Since

$$C_0 = 0$$

Calculate each carry.

Carry C_1

$$C_1$$

$$= 0 + (1 \times 0)$$

$$= 0$$

Carry C_2

$$C_2$$

$$= 1 + (0 \times 0) + (0 \times 1 \times 0)$$

$$= 1$$

Carry C_3

$$C_3$$

$$= 0 + (1 \times 1)$$

$$= 1$$

Carry C_4

$$C_4$$

$$= 0 + (1 \times 0)$$

$$\bullet (1 \times 1 \times 1)$$

$$= 1$$

Therefore,

Carry	Value
C_0	0
C_1	0
C_2	1
C_3	1
C_4	1

Step 4: Calculate the Sum

The sum equation is

$$S = P \oplus \text{Carry}$$

$$S_0$$

$$S_0$$

$$= P_0 \oplus C_0$$

$$= 1 \oplus 0$$

$$= 1$$

$$S_1$$

$$S_1$$

$$= 0 \oplus 0$$

$$= 0$$

$$S_2$$

$$S_2$$

$$= 1 \oplus 1$$

$$= 0$$

$$S_3$$

$$S_3$$

$$= 1 \oplus 1$$

$$= 0$$

Final Sum

0001

Final Carry

1

Therefore,

Result = 10001

Verification

Decimal value of

$$A = 1011$$

$$= 11$$

$$B = 0110$$

$$= 6$$

$$11 + 6$$

$$= 17$$

Binary of 17

$$= 10001$$

Hence,

The calculated result is correct.

Step 5: Comparison between Ripple Carry Adder and Carry Look-Ahead Adder

Ripple Carry Adder	Carry Look-Ahead Adder
Carry moves from one stage to the next.	All carries are calculated simultaneously.
High delay because every carry waits for the previous carry.	Very low delay because carries are generated in advance.
Simple hardware design.	More complex hardware design.
Suitable for small circuits.	Suitable for high-speed processors.
Slower performance.	Faster performance.

Step 6: Time Delay Analysis

Ripple Carry Adder

Suppose each full adder requires **1 unit of time**.

For a 4-bit adder,

Carry passes through four stages.

Total delay

=4 units

Carry Look-Ahead Adder

The carry values are calculated simultaneously.

The approximate delay is only

≈2 units

Therefore,

CLA reduces the addition time by almost **50%** compared to the Ripple Carry Adder.

Why CLA Improves Speed in Modern Processors

The Carry Look-Ahead Adder is faster because it does not wait for the carry to travel through each full adder. Instead, it uses **Generate** and **Propagate** signals to determine all carry bits at the same time. This reduces computation delay and increases the speed of arithmetic operations. Since processors perform millions of additions every second, reducing carry delay greatly improves overall system performance. Hence, modern CPUs and high-performance computers prefer the Carry Look-Ahead Adder over the Ripple Carry Adder.

Final Result

Parameter	Result
Binary Input A	1011
Binary Input B	0110
Generate Signals	$G_0=0, G_1=1, G_2=0, G_3=0$
Propagate Signals	$P_0=1, P_1=0, P_2=1, P_3=1$
Final Sum	10001
Time Delay (RCA)	4 Units
Time Delay (CLA)	2 Units
Faster Adder	Carry Look-Ahead Adder (CLA)

Interpretation of Results

The **Carry Look-Ahead Adder (CLA)** speeds up binary addition by calculating carry bits in advance using **Generate (G)** and **Propagate (P)** signals. It produced the correct result **10001 (17)** with less delay than a Ripple Carry Adder. Therefore, CLA is preferred in modern processors for its high speed and efficiency.

Question 3:

Step 1: Given Numbers

Multiplicand (M) = **-6**

Multiplier (Q) = **+5**

We use **4-bit 2's complement representation**.

Convert **-6** into Binary

First, write +6.

+6 = **0110**

Take 1's complement:

0110 $\rightarrow$ 1001

Add 1:

1001

+0001

1010

Therefore,

-6 = 1010

Convert +5 into Binary

+5 = **0101**

Initial Values

Register	Value
Multiplicand (M)	1010
Multiplier (Q)	0101
Accumulator (A)	0000
Booth Bit (Q_{-1})	0
Number of Bits	4

Step 2: Booth's Rules

The algorithm checks the last bit of the multiplier (Q_0) and the Booth bit (Q_{-1}).

$Q_0 Q_{-1}$	Operation
00	No operation
11	No operation
01	Add M to A
10	Subtract M from A

After each operation, perform **one arithmetic right shift**.

Step 3: Booth's Algorithm Execution

Iteration 1

Initial values

A = 0000

Q = 0101

$Q_{-1} = 0$

$Q_0 Q_{-1} = 10$

According to Booth's Rule,

Subtract M from A

Since

M = 1010 (-6)

Subtracting M means

$A = A - M$

$= 0 - (-6)$

= +6

Binary of +6

0110

Now,

A = 0110

Perform Arithmetic Right Shift

Before Shift

A = 0110

Q = 0101

$Q_{-1} = 0$

After Shift

A = 0011

Q = 0010

$Q_{-1} = 1$

Iteration 2

Current values

A = 0011

Q = 0010

$Q_{-1} = 1$

$Q_0 Q_{-1}$

= 01

Rule

Add M

A

= 0011

+ 1010

1101

Perform Arithmetic Right Shift

After Shift

A = 1110

Q = 1001

$Q_{-1} = 0$

Iteration 3

Current values

A = 1110

Q = 1001

$Q_{-1} = 0$

$Q_0 Q_{-1}$

$= 10$

Rule

Subtract M

Subtracting -6 is equivalent to adding $+6$.

A

1110

+0110

0100

Ignore carry.

Perform Arithmetic Right Shift

$A = 0010$

$Q = 0100$

$Q_{-1} = 1$

Iteration 4

Current values

$A = 0010$

$Q = 0100$

$Q_{-1} = 1$

$Q_0 Q_{-1}$

$= 01$

Rule

Add M

A

0010

+1010

1100

Perform Arithmetic Right Shift

$A = 1110$

$Q = 0010$

$Q_{-1} = 0$

Step 4: Final Product

The final product is obtained by combining

A and Q.

A = 1110

Q = 0010

Final Product

11100010

Step 5: Verify the Result

Decimal multiplication

-6×5

$= -30$

Now convert -30 into 8-bit binary.

+30

= 00011110

1's Complement

11100001

Add 1

11100010

Therefore,

$-30 = 11100010$

This matches the result obtained using Booth's Algorithm.

Hence,

Answer is Correct.

Role of Registers

Accumulator (A)

The accumulator stores the intermediate result after each addition or subtraction.

Multiplier (Q)

The multiplier contains the second operand and is shifted after every iteration.

Booth Bit (Q₋₁)

The Booth bit stores the previous least significant bit of the multiplier. It helps determine whether addition, subtraction, or no operation should be performed.

Role of Arithmetic Right Shift

After each iteration, the combined register (A, Q, Q₋₁) is shifted to the right.

The sign bit of A is preserved during the shift. This is called an **Arithmetic Right Shift**, and it ensures that negative numbers remain negative after shifting.

Advantages of Booth's Algorithm

1. It correctly handles both positive and negative numbers.
2. It reduces the number of addition and subtraction operations.
3. It is faster than ordinary binary multiplication.
4. It efficiently handles consecutive 1's in the multiplier.
5. It is widely used in modern processors and digital signal processors.

Comparison with Traditional Multiplication

Traditional Multiplication	Booth's Algorithm
Performs addition for every 1 in the multiplier.	Performs fewer additions and subtractions.
Slower for signed numbers.	Faster for signed numbers.
More computation time.	Less computation time.
Less efficient for long binary numbers.	More efficient for large binary numbers.

Final Result

Parameter	Value
Multiplicand (M)	-6 (1010)
Multiplier (Q)	+5 (0101)
Final Binary Product	11100010
Decimal Product	-30

Interpretation of Results

Booth's Algorithm correctly multiplied **-6** and **+5** using **2's complement representation**. The final result was **11100010** (**-30**). Since it uses fewer addition and subtraction operations than traditional multiplication, it is **faster and more efficient**, making it suitable for modern processors.

Question 4:

Step 1: Convert Decimal Numbers into Binary

Dividend = 13

13 = 1101₂

Divisor = 3

3 = **0011**₂

We use **4-bit registers**.

Initial values:

Register	Value
Dividend (Q)	1101
Divisor (M)	0011
Remainder (A)	0000
Number of Bits	4

PART A – Restoring Division Algorithm

Working Principle

In the restoring division algorithm:

1. Shift the remainder and quotient to the left.
2. Subtract the divisor from the remainder.
3. If the remainder is positive, write **1** in the quotient.
4. If the remainder becomes negative, **restore** the previous remainder by adding the divisor again and write **0** in the quotient.
5. Repeat the process until all bits are processed.

Iteration 1

Initial values

A = 0000

Q = 1101

Shift left

A = 0001

Q = 1010

Subtract divisor

0001

-0011

=1110 (Negative)

Since the remainder is negative, restore it.

1110

+0011

=0001

Quotient bit = **0**

Current values

A = 0001

Q = 1010

Iteration 2

Shift left

A = 0011

Q = 0100

Subtract divisor

0011

−0011

=0000

Remainder is positive.

Quotient bit = **1**

Current values

A = 0000

Q = 0101

Iteration 3

Shift left

A = 0000

Q = 1010

Subtract divisor

0000

−0011

=1101 (Negative)

Restore remainder

1101

+0011

=0000

Quotient bit = **0**

Current values

A = 0000

Q = 1010

Iteration 4

Shift left

A = 0001

Q = 0100

Subtract divisor

0001

-0011

=1110 (Negative)

Restore remainder

1110

+0011

=0001

Quotient bit = 0

Final Result (Restoring Division)

The binary quotient obtained is

0100₂

Binary remainder

0001₂

Convert into decimal.

Quotient

0100₂ = 4

Remainder

0001₂ = 1

Therefore,

13 ÷ 3 = 4

Remainder = 1

PART B – Non-Restoring Division Algorithm

Working Principle

The non-restoring division algorithm avoids restoring the remainder whenever it becomes negative.

The steps are:

1. Shift the registers to the left.
2. If the previous remainder is positive, subtract the divisor.
3. If the previous remainder is negative, add the divisor.
4. Continue the process until all bits are completed.
5. If the final remainder is negative, add the divisor once to obtain the correct remainder.

This method reduces the number of arithmetic operations.

Iteration 1

Initial values

A = 0000

Q = 1101

Shift left

A = 0001

Q = 1010

Subtract divisor

0001

−0011

= 1110 (Negative)

Quotient bit = 0

Iteration 2

Since the previous remainder is negative,

Add divisor

1110

+0011

= 0001

Shift left

A = 0011

Q = 0100

Subtract divisor

0011

−0011

= 0000

Quotient bit = 1

Iteration 3

Previous remainder is positive.

Shift left

A = 0000

Q = 1010

Subtract divisor

0000

−0011

= 1101 (Negative)

Quotient bit =0

Iteration 4

Previous remainder is negative.

Add divisor

1101

+0011

=0000

Shift left

A =0001

Q =0100

Subtract divisor

0001

-0011

=1110

Quotient bit =0

Since the final remainder is negative,

Restore it once.

1110

+0011

=0001

Final Result (Non-Restoring Division)

Quotient

0100₂

Remainder

0001₂

Decimal values

Quotient = **4**

Remainder = **1**

Therefore,

13 ÷ 3 = 4

Remainder = 1

Verification

Using decimal arithmetic,

$$13 \div 3$$

$$= 4$$

Remainder = 1

Therefore, both restoring and non-restoring division algorithms produce the correct result.

Comparison between Restoring and Non-Restoring Division

Restoring Division	Non-Restoring Division
Restores the remainder whenever it becomes negative.	Does not restore the remainder immediately.
Requires more addition operations.	Requires fewer arithmetic operations.
Slower execution.	Faster execution.
Simpler to understand.	Slightly more complex but more efficient.
Used in basic processors.	Used in modern high-speed processors.

Time Complexity Comparison

Parameter	Restoring	Non-Restoring
Shift Operations	4	4
Subtractions	4	4
Restoring Additions	More	Fewer
Execution Speed	Slower	Faster
Hardware Efficiency	Lower	Higher

Why Non-Restoring Division is Preferred

The non-restoring division algorithm is preferred because it avoids restoring the remainder after every negative result. Instead, it performs the correction only when required. This reduces the number of arithmetic operations, decreases execution time, and improves processor performance. Since modern processors execute millions of arithmetic operations every second, reducing unnecessary additions helps improve speed and hardware efficiency.

Final Result

Algorithm	Quotient	Remainder
Restoring Division	0100 ₂ (4)	0001 ₂ (1)
Non-Restoring Division	0100 ₂ (4)	0001 ₂ (1)

Interpretation of Results

Both Restoring and Non-Restoring Division correctly produced Quotient = 4 and Remainder = 1. However, Non-Restoring Division performs fewer operations, making it faster and more efficient, so it is preferred in modern processors.

Question 5:

Step 1: Convert the Decimal Number into Binary

Given number:

-13.25

Ignore the negative sign first.

Convert Integer Part (13)

$$13 \div 2$$

Division	Remainder
13 ÷ 2	1
6 ÷ 2	0
3 ÷ 2	1
1 ÷ 2	1

Reading from bottom,

$$13 = 1101_2$$

Convert Fractional Part (0.25)

Multiply by 2 repeatedly.

Calculation	Integer Part
$0.25 \times 2 = 0.50$	0
$0.50 \times 2 = 1.00$	1

Therefore,

$$0.25 = 0.01_2$$

Complete Binary Number

13.25

=

$$1101.01_2$$

Since the original number is negative,

$$-13.25 = -1101.01_2$$

Step 2: Normalize the Binary Number

IEEE 754 stores numbers in normalized scientific notation.

Move the binary point until only one digit remains before it.

$$1101.01_2$$

=

$$1.10101 \times 2^3$$

Therefore,

- Significand = **1.10101**
- Exponent = **3**

Step 3: IEEE 754 Single Precision (32-bit)

Single precision consists of:

Field	Number of Bits
Sign	1
Exponent	8
Mantissa	23

Total = **32 bits**

(a) Sign Bit

Since the number is negative,

Sign = 1

(b) Exponent

Actual exponent = **3**

Bias for single precision = **127**

Stored exponent

= $127 + 3$

= **130**

Convert 130 into binary.

130

=

10000010₂

Exponent field

=

10000010

(c) Mantissa

Normalized number

1.10101

The leading **1** is not stored.

Only the fractional part is stored.

Mantissa

=

101010000000000000000000

(remaining bits are filled with zeros)

Final IEEE 754 Single Precision Representation

Field	Value
Sign	1
Exponent	10000010
Mantissa	101010000000000000000000

Final 32-bit representation

1 10000010 101010000000000000000000

Step 4: IEEE 754 Double Precision (64-bit)

Double precision consists of:

Field	Number of Bits
Sign	1
Exponent	11
Mantissa	52

Total

64 bits

(a) Sign Bit

Negative number

Sign = 1

(b) Exponent

Bias for double precision = 1023

Stored exponent

$$1023+3 = 1026$$

1026 in binary = **10000000010₂**

Exponent field = 10000000010

(c) Mantissa

Mantissa = 1010100

(remaining bits are filled with zeros)

Final IEEE 754 Double Precision Representation

[illegible]

Final 64-bit representation

1 10000000010 10101000

Step 5: Floating-Point Addition

Consider two numbers

13.25

and

2.5

Binary Representation

$$13.25 = 1101.01_2$$

$$2.5 = 10.1_2$$

Normalize them.

$$13.25 = 1.10101 \times 2^3$$

$$2.5 = 1.01 \times 2^1$$

Step 6: Equalize the Exponents

The exponents must be equal before addition.

Difference

$$3 - 1 = 2$$

Shift the mantissa of the smaller number.

$$1.01 \times 2^1$$

↓

$$0.0101 \times 2^3$$

Now both numbers have exponent

$$2^3$$

Step 7: Add the Mantissas

$$\begin{array}{r} 1.10101 \\ + 0.01010 \\ \hline \end{array}$$

$$1.11111$$

Exponent remains

$$2^3$$

Step 8: Normalize the Result

The answer is already normalized.

Therefore,

$$\text{Result} = 1.11111 \times 2^3$$

Step 9: Convert Back to Decimal

Binary

$$1111.11_2 = 15.75$$

Therefore,

$$13.25 + 2.5 = 15.75$$

Normalization

Normalization means adjusting the binary number so that only one digit appears before the binary point.

Example

$$1101.01_2$$

↓

$$1.10101 \times 2^3$$

Normalization improves accuracy and allows floating-point numbers to have a standard representation.

Rounding

Sometimes the mantissa contains more bits than can be stored.

Extra bits are removed using rounding.

Common rounding method:

Round to Nearest Even

Example

$$1.101010111\dots$$

↓

$$1.101011$$

This reduces storage while keeping the value as accurate as possible.

Precision Loss

Floating-point numbers cannot exactly represent every decimal value.

For example,

$$0.1$$

cannot be represented exactly in binary.

Hence,

small rounding errors occur.

After performing many calculations, these small errors may accumulate and produce slight differences in the final result.

This is known as

Precision Loss.

Comparison between Single Precision and Double Precision

Feature	Single Precision	Double Precision
Total Bits	32	64
Sign Bits	1	1
Exponent Bits	8	11
Mantissa Bits	23	52
Precision	Lower	Higher
Memory Required	Less	More
Accuracy	Good	Excellent
Speed	Faster	Slightly Slower

Advantages of IEEE 754 Standard

1. Provides a standard method for representing floating-point numbers.
2. Supports very small and very large numbers.
3. Maintains good accuracy in scientific calculations.
4. Reduces calculation errors using normalization and rounding.
5. Widely used in modern processors, graphics systems, and scientific applications.

Final Result

[illegible]

Interpretation of Results

The **IEEE 754 standard** represents -13.25 using **sign, exponent, and mantissa** fields. **Single precision (32-bit)** uses less memory, while **double precision (64-bit)** provides higher accuracy. During floating-point addition, the processor aligns exponents, adds mantissas, and normalizes the result. Although small precision errors may occur, IEEE 754 ensures accurate and efficient floating-point calculations.